

Exercício Guiado: Testando PedidoService com Mock e Spy

Objetivo

Criar um serviço de pedidos que depende de um serviço de pagamento e testar sem executar pagamento real.

1. Criar o projeto

No terminal do VS Code:

```
mkdir aula-mocks-spies
cd aula-mocks-spies
npm init -y
```

2. Instalar dependências

```
npm install typescript jest ts-jest @types/jest ts-node-dev
@types/node -D
npx tsc --init
npx ts-jest config:init
```

3. Ajustar o package.json

Troque a parte de scripts por:

```
"scripts": {
  "test": "jest",
  "test:watch": "jest --watch"
}
```

4. Criar estrutura de pastas

```
src/
  services/
    PagamentoService.ts
    PedidoService.ts

tests/
  PedidoService.test.ts
```

5. Criar o PagamentoService

Arquivo:

src/services/PagamentoService.ts

Código:

```
export class PagamentoService {
  processarPagamento(valor: number): boolean {
    console.log(`Pagamento processado no valor de R$ ${valor}`);
    return true;
  }
}
```

Explicação

Esse serviço simula um pagamento real.

Na vida real, aqui poderia existir uma chamada para cartão, Pix, boleto, banco ou API externa.

6. Criar o PedidoService

Arquivo:

src/services/PedidoService.ts

Código:

```
import { PagamentoService } from '../PagamentoService';

export class PedidoService {
  constructor(private pagamentoService: PagamentoService) {}

  finalizarPedido(valor: number): string {
    if (valor <= 0) {
      throw new Error('Valor do pedido deve ser maior que zero');
    }

    const pagamentoAprovado =
      this.pagamentoService.processarPagamento(valor);

    if (!pagamentoAprovado) {
      throw new Error('Pagamento não aprovado');
    }

    return 'Pedido finalizado com sucesso';
  }
}
```

Explicação

O PedidoService depende do PagamentoService.

Ele faz três coisas:

Valida se o valor é maior que zero
Chama o serviço de pagamento
Retorna uma mensagem de sucesso

7. Criar o primeiro teste com Mock

Arquivo:

tests/PedidoService.test.ts

Código:

```
import { PedidoService } from '../src/services/PedidoService';

describe('PedidoService', () => {
  it('deve finalizar pedido quando pagamento for aprovado', () => {
    const mockPagamentoService = {
      processarPagamento: jest.fn().mockReturnValue(true)
    };

    const pedidoService = new PedidoService(mockPagamentoService as any);

    const resultado = pedidoService.finalizarPedido(100);

    expect(resultado).toBe('Pedido finalizado com sucesso');

    expect(mockPagamentoService.processarPagamento).toHaveBeenCalledTimes(1);
  });
});
```

Explicação

Aqui não usamos o pagamento real.

Criamos um objeto falso:

```
const mockPagamentoService = {
  processarPagamento: jest.fn().mockReturnValue(true)
};
```

Isso significa:

“Quando chamar `processarPagamento`, finja que deu certo.”

8. Rodar o teste

No terminal:

npm test

Resultado esperado:

PASS

9. Criar teste de erro com valor inválido

Adicione dentro do mesmo describe:

```
it('deve lançar erro quando valor do pedido for zero', () => {
  const mockPagamentoService = {
    processarPagamento: jest.fn()
  };

  const pedidoService = new PedidoService(mockPagamentoService as any);

  expect(() => pedidoService.finalizarPedido(0)).toThrow(
    'Valor do pedido deve ser maior que zero'
  );

  expect(mockPagamentoService.processarPagamento).not.toHaveBeenCalled();
});
```

Explicação

Esse teste garante duas coisas:

- O sistema lança erro se o valor for zero
- O pagamento nem chega a ser chamado

Isso é muito importante em sistemas reais.

10. Criar teste de pagamento recusado

```
it('deve lançar erro quando pagamento não for aprovado', () => {
  const mockPagamentoService = {
    processarPagamento: jest.fn().mockReturnValue(false)
  };

  const pedidoService = new PedidoService(mockPagamentoService as any);

  expect(() => pedidoService.finalizarPedido(100)).toThrow(
    'Pagamento não aprovado'
  );

  expect(mockPagamentoService.processarPagamento).toHaveBeenCalledWith(100);
});
```

Explicação

Aqui simulamos que o pagamento foi recusado.

Não precisamos de banco, cartão ou API externa.

O mock controla o cenário.

11. Exemplo com Spy

Agora vamos observar o comportamento real.

Adicione no teste:

```
import { PagamentoService } from '../src/services/PagamentoService';
```

Depois adicione este teste:

```
it('deve observar se o pagamento real foi chamado usando spy', () => {
  const pagamentoService = new PagamentoService();
  const spy = jest.spyOn(pagamentoService, 'processarPagamento');

  const pedidoService = new PedidoService(pagamentoService);

  const resultado = pedidoService.finalizarPedido(150);

  expect(resultado).toBe('Pedido finalizado com sucesso');
  expect(spy).toHaveBeenCalledWith(150);
});
```

Explicação

Aqui usamos o serviço real.

O `spy` apenas observa se o método foi chamado.

Ele não substitui o comportamento.

12. Arquivo final completo do teste

```
import { PedidoService } from '../src/services/PedidoService';
import { PagamentoService } from '../src/services/PagamentoService';

describe('PedidoService', () => {
  it('deve finalizar pedido quando pagamento for aprovado', () => {
    const mockPagamentoService = {
      processarPagamento: jest.fn().mockReturnValue(true)
    };

    const pedidoService = new PedidoService(mockPagamentoService as any);
```

```

    const resultado = pedidoService.finalizarPedido(100);

    expect(resultado).toBe('Pedido finalizado com sucesso');

    expect(mockPagamentoService.processarPagamento).toHaveBeenCalledWith(100);
  });

  it('deve lançar erro quando valor do pedido for zero', () => {
    const mockPagamentoService = {
      processarPagamento: jest.fn()
    };

    const pedidoService = new PedidoService(mockPagamentoService as any);

    expect(() => pedidoService.finalizarPedido(0)).toThrow(
      'Valor do pedido deve ser maior que zero'
    );

    expect(mockPagamentoService.processarPagamento).not.toHaveBeenCalled();
  });

  it('deve lançar erro quando pagamento não for aprovado', () => {
    const mockPagamentoService = {
      processarPagamento: jest.fn().mockReturnValue(false)
    };

    const pedidoService = new PedidoService(mockPagamentoService as any);

    expect(() => pedidoService.finalizarPedido(100)).toThrow(
      'Pagamento não aprovado'
    );

    expect(mockPagamentoService.processarPagamento).toHaveBeenCalledWith(100);
  });

  it('deve observar se o pagamento real foi chamado usando spy', () => {
    const pagamentoService = new PagamentoService();
    const spy = jest.spyOn(pagamentoService, 'processarPagamento');

    const pedidoService = new PedidoService(pagamentoService);

    const resultado = pedidoService.finalizarPedido(150);

    expect(resultado).toBe('Pedido finalizado com sucesso');
    expect(spy).toHaveBeenCalledWith(150);
  });
});

```

13. Desafio:

Criar uma nova regra:

Se o valor do pedido for maior que 1000, lançar erro:

```
Valor acima do limite permitido
```

Alterar o PedidoService

```
if (valor > 1000) {  
  throw new Error('Valor acima do limite permitido');  
}
```

Criar o teste esperado

```
it('deve lançar erro quando valor do pedido for maior que 1000', () =>  
{  
  const mockPagamentoService = {  
    processarPagamento: jest.fn()  
  };  
  
  const pedidoService = new PedidoService(mockPagamentoService as  
any);  
  
  expect(() => pedidoService.finalizarPedido(1500)).toThrow(  
    'Valor acima do limite permitido'  
  );  
  
  expect(mockPagamentoService.processarPagamento).not.toHaveBeenCalled()  
  ;  
});
```